2a. I'm using Claude and ChatGPT

2b. I'm accessing them on my laptop.

2c. Here is the prompt I used:

Generate C++ code using objects, not functions, to implement the following requirements:

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. **Then use that MaxHeap to handle all your email prioritizing for the CEO.**
- You must create objects from scratch. Do not include pre-existing heap modules

Test File format:

Command: EMAIL <sender category>, <subject line>, <date>

- Description:

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:
• Boss – read first
• Subordinate – read next
• Peer – read next
• ImportantPerson – read next
• OtherPerson – read last
If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.
EMAIL is followed by space. The rest of the fields are delimited.
Assume <sender category> is one of the five strings listed above.
Assume <subject line> is a string which may contain spaces, but not commas
Assume <date> is in the format: MM-DD-YYYY

Command: NEXT

- Description:

Next email for the CEO to read. Display the information on the terminal in the following format:
Sender:
Subject:
Date:

Command: READ

- Description:

CEO has read email and dealt with it

Command: COUNT

- Description:

Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it)

Sample Test File:

```
EMAIL Peer, Can you help me on this?,12-01-2024
EMAIL OtherPerson, Try our product,12-19-2024
EMAIL Boss, Important,12-20-2024
EMAIL Subordinate, How do I handle this?,12-25-2024
EMAIL ImportantPerson, Health Insurance Enrollment,12-31-2024
EMAIL Boss, Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

Your program should display this:

```
There are 6 emails to read.

Next email:
        Sender: Boss
        Subject: Never Mind
        Date: 01-03-2025
Next email:
        Sender: Boss
        Subject: Important
        Date: 12-20-2024

There are 4 emails to read
```

2d. (AI Prompt)

ChatGPT

```
#include <iostream>
#include <string>
#include <fstream>
#include <sstream>

using namespace std;


// ============================================================
// Email Object
// ============================================================
class Email
{
private:
    string sender;
    string subject;
    string date;

public:
    Email()
    {
        sender = "";
        subject = "";
        date = "";
    }

    Email(string s, string sub, string d)
    {
        sender = s;
        subject = sub;
        date = d;
    }

    string getSender() const
    {
        return sender;
    }

    string getSubject() const
    {
        return subject;
    }
```

```
string getDate() const
{
    return date;
}

// Returns the priority value for the sender category.
int getSenderPriority() const
{
    if (sender == "Boss")
        return 5;
    else if (sender == "Subordinate")
        return 4;
    else if (sender == "Peer")
        return 3;
    else if (sender == "ImportantPerson")
        return 2;
    else
        return 1;
}

// Convert MM-DD-YYYY into an integer that can be compared.
int getDateValue() const
{
    int month = stoi(date.substr(0, 2));
    int day = stoi(date.substr(3, 2));
    int year = stoi(date.substr(6, 4));

    return year * 10000 + month * 100 + day;
}

// Determines whether this email has higher priority
// than another email.
bool hasHigherPriority(const Email& other) const
{
    // Sender category has the highest importance.
    if (getSenderPriority() != other.getSenderPriority())
    {
        return getSenderPriority() > other.getSenderPriority();
    }

    // If sender categories are equal, newer email wins.
    return getDateValue() > other.getDateValue();
}
```

```
    void display() const
    {
        cout << "Sender: " << sender << endl;
        cout << "Subject: " << subject << endl;
        cout << "Date: " << date << endl;
    }
};


// ============================================================
// MaxHeap Object
// List-based implementation
// ============================================================
class MaxHeap
{
private:
    Email* heap;
    int capacity;
    int heapSize;

    void resize()
    {
        int newCapacity = capacity * 2;

        Email* newHeap = new Email[newCapacity];

        for (int i = 0; i < heapSize; i++)
        {
            newHeap[i] = heap[i];
        }

        delete[] heap;

        heap = newHeap;
        capacity = newCapacity;
    }

    void swapEmails(int first, int second)
    {
        Email temp = heap[first];
        heap[first] = heap[second];
        heap[second] = temp;
    }
```

```
void bubbleUp(int index)
{
    while (index > 0)
    {
        int parent = (index - 1) / 2;

        if (heap[index].hasHigherPriority(heap[parent]))
        {
            swapEmails(index, parent);
            index = parent;
        }
        else
        {
            break;
        }
    }
}

void bubbleDown(int index)
{
    while (true)
    {
        int leftChild = 2 * index + 1;
        int rightChild = 2 * index + 2;

        int largest = index;

        if (leftChild < heapSize &&
            heap[leftChild].hasHigherPriority(heap[largest]))
        {
            largest = leftChild;
        }

        if (rightChild < heapSize &&
            heap[rightChild].hasHigherPriority(heap[largest]))
        {
            largest = rightChild;
        }

        if (largest != index)
        {
            swapEmails(index, largest);
            index = largest;
```

```
            }
            else
            {
                break;
            }
        }
    }

public:
    MaxHeap()
    {
        capacity = 10;
        heapSize = 0;
        heap = new Email[capacity];
    }

    ~MaxHeap()
    {
        delete[] heap;
    }

    bool isEmpty() const
    {
        return heapSize == 0;
    }

    int size() const
    {
        return heapSize;
    }

    // Insert an email into the MaxHeap.
    void insert(const Email& email)
    {
        if (heapSize == capacity)
        {
            resize();
        }

        heap[heapSize] = email;

        bubbleUp(heapSize);

        heapSize++;
```

```
    }

    // IMPORTANT:
    // Returns the highest-priority email WITHOUT deleting it.
    // This is what allows two NEXT commands in a row
    // to display the same email.
    Email peek() const
    {
        return heap[0];
    }

    // Removes the highest-priority email.
    Email removeMax()
    {
        Email highestPriority = heap[0];

        heapSize--;

        if (heapSize > 0)
        {
            heap[0] = heap[heapSize];
            bubbleDown(0);
        }

        return highestPriority;
    }
};


// ============================================================
// CEO Inbox Object
// ============================================================
class CEOInbox
{
private:
    MaxHeap emails;

public:

    void addEmail(string sender, string subject, string date)
    {
        Email newEmail(sender, subject, date);
        emails.insert(newEmail);
    }
```

```
    void nextEmail()
    {
        if (emails.isEmpty())
        {
            return;
        }

        Email next = emails.peek();

        cout << "Next email:" << endl;
        next.display();
    }

    void readEmail()
    {
        if (emails.isEmpty())
        {
            return;
        }

        emails.removeMax();
    }

    void countEmails()
    {
        cout << "There are "
             << emails.size()
             << " emails to read."
             << endl;
    }

    bool isEmpty() const
    {
        return emails.isEmpty();
    }
};


// ============================================================
// Main Program
// ============================================================
int main()
{
```

```
CEOInbox inbox;

string line;

// Read commands from standard input.
while (getline(cin, line))
{
    if (line.empty())
    {
        continue;
    }

    // ------------------------------------------------
    // EMAIL command
    // ------------------------------------------------
    if (line.substr(0, 5) == "EMAIL")
    {
        // Remove "EMAIL "
        string information = line.substr(6);

        // Find first comma.
        size_t firstComma = information.find(',');

        // Find second comma.
        size_t secondComma = information.find(',', firstComma + 1);

        // Extract sender.
        string sender =
            information.substr(0, firstComma);

        // Extract subject.
        string subject =
            information.substr(
                firstComma + 2,
                secondComma - firstComma - 2
            );

        // Extract date.
        string date =
            information.substr(secondComma + 1);

        inbox.addEmail(sender, subject, date);
    }
```

```
        // ------------------------------------------------
        // NEXT command
        // ------------------------------------------------
        else if (line == "NEXT")
        {
            inbox.nextEmail();
        }

        // ------------------------------------------------
        // READ command
        // ------------------------------------------------
        else if (line == "READ")
        {
            inbox.readEmail();
        }

        // ------------------------------------------------
        // COUNT command
        // ------------------------------------------------
        else if (line == "COUNT")
        {
            inbox.countEmails();
        }
    }

    return 0;
}
```

Claude

```
// email_prioritizer.cpp
// Prioritizes a CEO's inbox using a hand-built, list-based MaxHeap.
//
// Build: g++ -std=c++17 -Wall -o email_prioritizer email_prioritizer.cpp
// Run:   ./email_prioritizer test.txt      (or pipe the file into stdin)

#include <fstream>
#include <iostream>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>

// ---------------------------------------------------------------------------
```

```
// Email: one message in the inbox, and the rules for comparing priority.
// ------------------------------------------------------------------------
class Email {
public:
    Email(const std::string& sender, const std::string& subject,
          const std::string& date, long arrivalOrder)
        : sender_(sender),
          subject_(subject),
          date_(date),
          categoryRank_(rankForCategory(sender)),
          dateKey_(keyForDate(date)),
          arrivalOrder_(arrivalOrder) {}

    // Higher sender category wins; within a category, the newer date wins.
    // If both match, the email that arrived later in the file wins.
    bool hasHigherPriorityThan(const Email& other) const {
        if (categoryRank_ != other.categoryRank_) {
            return categoryRank_ > other.categoryRank_;
        }
        if (dateKey_ != other.dateKey_) {
            return dateKey_ > other.dateKey_;
        }
        return arrivalOrder_ > other.arrivalOrder_;
    }

    void display(std::ostream& out) const {
        out << "Sender: " << sender_ << "\n"
            << "Subject: " << subject_ << "\n"
            << "Date: " << date_ << "\n";
    }

private:
    std::string sender_;
    std::string subject_;
    std::string date_;
    int categoryRank_;
    long dateKey_;
    long arrivalOrder_;

    static int rankForCategory(const std::string& category) {
        if (category == "Boss")            return 5;
        if (category == "Subordinate")     return 4;
        if (category == "Peer")            return 3;
        if (category == "ImportantPerson") return 2;
```

```
        if (category == "OtherPerson")     return 1;
        return 0;  // unknown categories sort to the very end
    }

    // Converts MM-DD-YYYY into YYYYMMDD so dates compare as plain numbers.
    static long keyForDate(const std::string& date) {
        if (date.size() < 10) {
            return 0;
        }
        try {
            long month = std::stol(date.substr(0, 2));
            long day   = std::stol(date.substr(3, 2));
            long year  = std::stol(date.substr(6, 4));
            return year * 10000 + month * 100 + day;
        } catch (const std::exception&) {
            return 0;
        }
    }
};

// ---------------------------------------------------------------------------
// MaxHeap: list-based (array-backed) binary max-heap of Emails.
// For index i: parent = (i-1)/2, left child = 2i+1, right child = 2i+2.
// ---------------------------------------------------------------------------
class MaxHeap {
public:
    void insert(const Email& email) {
        items_.push_back(email);
        siftUp(items_.size() - 1);
    }

    const Email& peekMax() const {
        if (isEmpty()) {
            throw std::out_of_range("peekMax called on an empty heap");
        }
        return items_[0];
    }

    void removeMax() {
        if (isEmpty()) {
            return;
        }
        items_[0] = items_.back();
        items_.pop_back();
```

```
        if (!isEmpty()) {
            siftDown(0);
        }
    }

    std::size_t size() const { return items_.size(); }
    bool isEmpty() const { return items_.empty(); }

private:
    std::vector<Email> items_;

    static std::size_t parentOf(std::size_t i) { return (i - 1) / 2; }
    static std::size_t leftOf(std::size_t i)   { return 2 * i + 1; }
    static std::size_t rightOf(std::size_t i)  { return 2 * i + 2; }

    void siftUp(std::size_t index) {
        while (index > 0) {
            std::size_t parent = parentOf(index);
            if (!items_[index].hasHigherPriorityThan(items_[parent])) {
                break;
            }
            swapItems(index, parent);
            index = parent;
        }
    }

    void siftDown(std::size_t index) {
        const std::size_t count = items_.size();
        while (true) {
            std::size_t left = leftOf(index);
            std::size_t right = rightOf(index);
            std::size_t best = index;

            if (left < count && items_[left].hasHigherPriorityThan(items_[best])) {
                best = left;
            }
            if (right < count && items_[right].hasHigherPriorityThan(items_[best])) {
                best = right;
            }
            if (best == index) {
                break;
            }
            swapItems(index, best);
            index = best;
```

```
        }
    }

    void swapItems(std::size_t a, std::size_t b) {
        Email temp = items_[a];
        items_[a] = items_[b];
        items_[b] = temp;
    }
};

// ---------------------------------------------------------------------------
// CEOInbox: the priority queue of unread emails, built on the MaxHeap.
// ---------------------------------------------------------------------------
class CEOInbox {
public:
    void receive(const Email& email) { heap_.insert(email); }

    // Shows the top email without removing it, so repeated NEXTs match.
    void showNext(std::ostream& out) const {
        if (heap_.isEmpty()) {
            out << "No emails to read.\n";
            return;
        }
        out << "Next email:\n";
        heap_.peekMax().display(out);
    }

    // Removes the top email whether or not it was displayed first.
    void markRead() { heap_.removeMax(); }

    void showCount(std::ostream& out) const {
        out << "There are " << heap_.size() << " emails to read.\n";
    }

private:
    MaxHeap heap_;
};

// ---------------------------------------------------------------------------
// CommandProcessor: reads the test file and drives the inbox.
// ---------------------------------------------------------------------------
class CommandProcessor {
public:
    CommandProcessor(CEOInbox& inbox, std::ostream& out)
```

```
        : inbox_(inbox), out_(out), arrivalCounter_(0), lastOutputWasNext_(false) {}

    void run(std::istream& in) {
        std::string line;
        while (std::getline(in, line)) {
            line = trim(line);
            if (!line.empty()) {
                process(line);
            }
        }
    }

private:
    CEOInbox& inbox_;
    std::ostream& out_;
    long arrivalCounter_;
    bool lastOutputWasNext_;  // keeps blank-line spacing matching the sample

    void process(const std::string& line) {
        const std::string emailPrefix = "EMAIL ";
        if (line.compare(0, emailPrefix.size(), emailPrefix) == 0) {
            handleEmail(line.substr(emailPrefix.size()));
        } else if (line == "NEXT") {
            inbox_.showNext(out_);
            lastOutputWasNext_ = true;
        } else if (line == "READ") {
            inbox_.markRead();
        } else if (line == "COUNT") {
            if (lastOutputWasNext_) {
                out_ << "\n";
            }
            inbox_.showCount(out_);
            out_ << "\n";
            lastOutputWasNext_ = false;
        } else {
            std::cerr << "Unrecognized command: " << line << "\n";
        }
    }

    // Parses "<sender category>, <subject line>, <date>".
    void handleEmail(const std::string& fields) {
        std::vector<std::string> parts;
        std::stringstream stream(fields);
        std::string piece;
```

```
        while (std::getline(stream, piece, ',')) {
            parts.push_back(trim(piece));
        }
        if (parts.size() != 3) {
            std::cerr << "Skipping malformed EMAIL line: " << fields << "\n";
            return;
        }
        inbox_.receive(Email(parts[0], parts[1], parts[2], arrivalCounter_++));
    }

    static std::string trim(const std::string& text) {
        const std::string whitespace = " \t\r\n";
        std::size_t start = text.find_first_not_of(whitespace);
        if (start == std::string::npos) {
            return "";
        }
        std::size_t end = text.find_last_not_of(whitespace);
        return text.substr(start, end - start + 1);
    }
};

// ---------------------------------------------------------------------------
// Entry point: wire up the objects and hand them the input stream.
// ---------------------------------------------------------------------------
int main(int argc, char* argv[]) {
    CEOInbox inbox;
    CommandProcessor processor(inbox, std::cout);

    if (argc > 1) {
        std::ifstream file(argv[1]);
        if (!file) {
            std::cerr << "Could not open file: " << argv[1] << "\n";
            return 1;
        }
        processor.run(file);
    } else {
        processor.run(std::cin);
    }
    return 0;
}
```

2e.

Correctness: Overall, both Gen AIs executed the prompt as stated. Both programs used the Max heap class specified in the instructions and placed emails in the correct priority order. However, neither program exactly matched the expected output format in the instructions. Because I neglected to indent the example output properly, both programs combined the information instead of indenting and separating it correctly. I'll need to address this in my final code. The biggest issue is the subject-line output. Claude performed correctly, but ChatGPT omitted the first letter of every subject line. For example, "Never Mind" appeared as "ever Mind," showing that ChatGPT did not print the prompt correctly. The code assumes there is a space and skips two characters after the first one in the test prompt. ChatGPT would also fail an edge-case test involving badly formatted dates. Therefore, Claude passed the correctness test, while ChatGPT had several issues and did not fully meet the requirements.

Execution Time: Both programs have the same big O for command execution. For example, adding an email and removing it from the top involve O(log n); looking at the next email and counting emails take O(1); and when both heaps grow, they use O(1). The real difference in execution time appears when the email command is used. When the code sees an email command in the file, its class calculates the category rank and date number. Both codes do this, and the email is called again to compare with another email to determine which has higher priority; this is where the real issue lies. Claud stores emails and compares them based on the stored numbers, but ChatGPT recalculates both values every time it compares emails, which increases overall execution time.

Space Complexity: Both programs use O(n) space. Claude uses a vector to manage storage and stores three extra numbers per email: rank, date number, and arrival order. Therefore, Claude only doubles its space when needed, which is similar to how ChatGPT approaches storage. However, it manually copies everything over and deletes the old ones each time it fills up.

Maintainability: Claude's program is divided into four classes (email, max heap, CEO inbox, and command processor). Each class has a descriptive name and a heading comment, but it lacks a prologue and needs more comments to support future maintenance. ChatGPT also uses classes and has clear section banners. However, its commands are located inside main rather than in an object, which means it can only be read from standard input and requires a special run prompt (./chatgpt < test.txt). Additionally, ChatGPT handles its own memory bookkeeping instead of letting C++ handle it, which creates maintainability issues because it lacks a guaranteed, reliable way to store information. Additionally, ChatGPT's code has no prologue and few comments, which reduces overall maintainability.

2f. Out of the two Gen AIs, Claude is my pick. While it still has some errors overall, its code was much cleaner than ChatGPT's in terms of correctness and maintainability. For correctness, it never cut out letters and executed the code as the prompt instructed. As for maintainability, it

doesnt have the problem of requiring the standard input to be used like ChatGPT and overall is much more orgranized in it’s code than ChatGPT.

2g.

Correctness: For the improved code, I first fixed the formatting issue, so it displays the correct output when the code is written alongside the test text file. Everything is now tabbed and indented correctly and matches the proper output. Additionally, I removed the "no emails to read" message because the instructions never required it.

Execution Time: As for the execution time, there was no real need to fix it, since both programs didn't encounter any time issues and ran for the same length of time. However, I did modify a bit of code that could save a small fraction of time. Specifically, I removed the flag that determined whether a blank line needed to be printed. Now, every output follows one rule, which reduces the overall time. While this isn't a huge change, it saves a bit of time.

Space Complexity: As for space complexity, nothing has changed. The vector is already an efficient way to save space when it needs to expand without wasting too much storage. Also, since this process isn't done by hand, much like ChatGPT, there's less room for error. Removing the flag I mentioned earlier from execution time section also saves a little storage.

Maintainability: The maintainability had a couple of updates, including a brief heading comment and a full prologue that includes the project name, description, inputs, outputs, collaborative sources, authors, and dates and revisions I made. Additionally, I added comments on each class explaining the purpose of that class, and I added comments on each line to detail what each line's purpose is, which is good for maintainability in the professional field. If someone else looks at the code, they know exactly how it operates and what to change if an update is needed.